

# Classical Aerial Single-Object Tracking via CSRT with MobileNet-V3-Small Appearance Verification

Team AIEEEs · MTC-AIC4 Phase I · Single-Object Aerial Tracking

Military Technical College Artificial Intelligence Competition 2026

---

## 1. Abstract

---

We present a classical, training-free aerial single-object tracker that combines the Channel and Spatial Reliability Tracker (CSRT) with an off-the-shelf MobileNet-V3-Small appearance verifier operating directly on ImageNet-pretrained weights. No task-specific fine-tuning is performed and no learned checkpoints are shipped. The system is designed to run within a strict 25 ms per-frame latency budget on a single GPU and produces a Kaggle-format submission CSV covering all frames at the original acquisition frame rate.

## 2. Introduction

---

Aerial single-object tracking is challenging due to small target size, abrupt camera motion, occlusions, and strict real-time requirements imposed by embedded platforms. Deep-learning trackers achieve state-of-the-art accuracy but require expensive training pipelines and large checkpoints. Our goal was to build a robust, reproducible baseline using only classical algorithms augmented with lightweight pretrained features — delivering competitive performance without any training data dependency or checkpoint management overhead.

The core insight is that MobileNet-V3-Small, when used purely as a fixed feature extractor (ImageNet weights, no fine-tuning), provides strong enough appearance discriminability for appearance-verification and template-diversity decisions, while CSRT handles the frame-to-frame localisation. This keeps the system fully reproducible: the exact same behaviour is obtained on any machine with a standard torchvision installation.

## 3. Method

---

### 3.1 Tracking backbone — CSRT

OpenCV's CSRT tracker serves as the primary localisation engine. Channel and spatial reliability maps weight each HOG/Colour channel independently, making it robust to partial occlusion and background clutter. CSRT is initialised once per sequence with the provided ground-truth bounding box and re-initialised whenever our appearance verifier detects target recovery after a lost phase. We run CSRT with channel weights enabled and three ADMM optimisation iterations to balance accuracy against speed.

### 3.2 Appearance model — MobileNet-V3-Small (fixed pretrained)

MobileNet-V3-Small is loaded with default ImageNet pretrained weights from torchvision. The classifier head is replaced with an Identity layer, exposing the 576-dimensional penultimate feature vector. All weights are frozen — **no fine-tuning is performed**. Each candidate patch is resized to 256×256, normalised with ImageNet statistics, and passed through the frozen backbone. The output embedding is L2-normalised before cosine-similarity comparisons.

MobileNet is invoked selectively: (i) every third frame during successful tracking, (ii) during grid-search recovery, and (iii) skipped entirely for tiny objects (area < 0.5% of the frame) and large objects (> 10%) where the latency budget is tightest or the appearance signal is least informative.

### 3.3 Component overview

- **Template Bank.** Stores up to 8 diverse appearance templates (cosine-distance threshold 0.12 for diversity). The original initialisation template is always preserved to resist drift.
- **Velocity Estimator.** A 6-frame rolling window tracks bounding-box centre motion. The resulting velocity is used for Kalman-style box prediction during lost phases and to weight the grid-search centre point.
- **Scale Controller.** Per-frame scale changes are clamped to  $\pm 8\%$ , preventing runaway box growth caused by CSRT overfitting to background regions.

- **Out-of-View Manager.** Records which frame border the target last exited from and prioritises border-region sweeps on re-entry, up to 40 frames after the exit event.
- **Grid Search.** When CSRT confidence falls below 0.45, a spatial grid (3×3 to 7×7 depending on lost duration) centred on the velocity-predicted position is searched using MobileNet similarity scores.
- **Dynamic Reinit Thresholds.** The CSRT re-initialisation score threshold drops from 0.60 to 0.40 after 20 lost frames and to 0.30 after 50, enabling recovery in severely degraded sequences.
- **Freeze Mode.** After 15 consecutive lost frames the tracker switches to pure Kalman propagation to avoid expensive searches on clearly occluded targets.
- **Object-Size Profiles.** Tiny/medium/large profiles tune the search grid, throttle rate, and MobileNet usage to stay within the 25 ms latency budget across a wide range of target sizes.

## 4. Datasets

We evaluated on the competition release datasets provided by MTC-AIC4. Two dataset splits were supplied:

- **dataset1** — test sequences; only initialisation bounding boxes are provided (first-frame annotation). No ground truth is available for evaluation, so predictions are submitted directly.
- **dataset2 / additional datasets** — full-annotation sequences used for internal validation where available. IoU and centre-error metrics are computed on these splits.

Since our method uses no training data (only pretrained ImageNet weights) the datasets are used solely for evaluation. No data augmentation, model selection, or hyper-parameter search was conducted on the competition data.

## 5. Implementation Details

|                              |           |                                                 |
|------------------------------|-----------|-------------------------------------------------|
|                              |           |                                                 |
| CSRT confidence threshold    | 0.45      | Minimum score to accept CSRT output             |
| Template bank size           | 8         | Max stored appearance templates                 |
| Template diversity threshold | 0.12      | Min cosine distance for new template            |
| Max scale change / frame     | 1.08 (8%) | Prevent box size explosion                      |
| Velocity window              | 6 frames  | Rolling-window velocity estimate                |
| Grid search size (lost)      | 5×5 – 7×7 | Spatial search after loss                       |
| Feature update interval      | 3 frames  | MobileNet call frequency (normal tracking)      |
| Latency budget               | 25 ms     | Hard per-frame budget; skips search if exceeded |
| Freeze threshold             | 15 frames | Switch to pure Kalman after N lost frames       |

Table 1. Key hyper-parameter settings.

## 6. Efficiency Analysis

|                                        |                           |
|----------------------------------------|---------------------------|
|                                        |                           |
| Backbone parameters                    | ~2.54 M                   |
| Backbone FLOPs (256×256 patch)         | ~0.06 GFLOPs              |
| Model disk size (fp32)                 | ~9.8 MB                   |
| Avg. inference latency (GPU)           | < 25 ms / frame           |
| MobileNet calls (normal tracking)      | Every 3rd frame           |
| MobileNet calls (tiny / large objects) | Disabled (CSRT only)      |
| No. learned checkpoints                | 0 — ImageNet weights only |

Table 2. Efficiency and model size metrics.

The tracker's efficiency stems from three design choices. First, MobileNet is called selectively rather than every frame, reducing the appearance-verification overhead by approximately 67 % during normal tracking. Second, a hard latency budget of 25 ms terminates grid search early if the frame budget is exceeded, guaranteeing real-time operation even on slower hardware. Third, tiny and large objects bypass MobileNet entirely, as empirical testing showed the CSRT confidence signal alone was sufficient for those size regimes while yielding substantial latency savings.

## 7. Results

The tracker processes all competition sequences at the original frame rate and produces predictions in the required CSV format (*id*, *x*, *y*, *w*, *h* per frame, where *id* is `<dataset>/<sequence>_<frame_index>`). Our final submission file `final_submission.csv` is included in the repository under `submission/`.

On internal validation sequences (dataset2) the tracker achieves competitive IoU scores without any task-specific training. Exact per-sequence numbers are available in the `all_results.csv` artefact generated during inference. The key operational metrics validated during development are summarised in Table 2 above.

## 8. Repository and Reproducibility

The full inference pipeline is containerised in a single Dockerfile based on `pytorch/pytorch:2.2.0-cuda12.1-cudnn8-runtime`. MobileNet-V3-Small weights are pre-downloaded at image build time via `torchvision`, so inference runs fully offline with no checkpoint downloads.

Minimal inference command:

|               |                                                                                                                                                                                                                                                          |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               |                                                                                                                                                                                                                                                          |
| Build image   | <code>docker build -t aieees-tracker .</code>                                                                                                                                                                                                            |
| Run inference | <code>docker run --rm --gpus all \ -v /path/to/data:/data \ -v /path/to/results:/results \ aieees-tracker \ python track.py \ --dataset_root /data/contest_release \ --save_root /results \ --config configs/tracker_config.yaml \ --generate_csv</code> |

Table 3. Docker-based inference commands (no checkpoint needed).

The `--weights` argument in `track.py` is fully optional (default `None`). When omitted, the feature extractor loads standard ImageNet pretrained weights automatically via `torchvision` — no file download is required from the user.

## 9. Conclusion

We demonstrated that a carefully engineered classical tracker — CSRT augmented with fixed pretrained MobileNet-V3-Small features and a suite of heuristic sub-components — can provide robust aerial single-object tracking within a real-time latency envelope without requiring any task-specific training. The approach is fully reproducible from a standard `torchvision` installation, ships zero learned checkpoints, and is straightforward to deploy via Docker.